计算机组成与设计
习题

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

HW1

Problem 1

1.14 Another pitfall cited in Section 1.11 is expecting to improve the overall performance of a computer by improving only one aspect of the computer. Consider a computer running a program that requires 250 s, with 70 s spent executing FP instructions, 85 s executed L/S instructions, and 40 s spent executing branch instructions.

1.14.1 [5] <§1.11> By how much is the total time reduced if the time for FP operations is reduced by 20%?

1.14.2 [5] <§1.11> By how much is the time for INT operations reduced if the total time is reduced by 20%?

1.14.3 [5] <§1.11> Can the total time can be reduced by 20% by reducing only the time for branch instructions?

1.13

1.13.1 $T_{fp} = 70 \times 0.8 = 56$ s. $T_{new} = 56 + 85 + 55 + 40 = 236$ s. Reduction: 5.6%

1.13.2 $T_{new} = 250 \times 0.8 = 200$ s, $T_{fp} + T_{l/s} + T_{branch} = 165$ s, $T_{int} = 35$ s. Reduction time INT: 58.8%

1.13.3 $T_{new} = 250 \times 0.8 = 200$ s, $T_{fp} + T_{int} + T_{l/s} = 210$ s. NO

HW2

Problem 1

Add comments to the following code and describe in one sentence what it computes.
Assume that a0 is used for the input and initially contains n, a positive integer.
Assume that a0 is used for the output.
Code for Problem 1:

```
begin:    addi t0, x0, 0          # t0 = 0
          addi t1, x0, 1          # t1 = 1
loop:     slt t2, a0, t1          # t2 = a0 < t1 ? 1 : 0
          bne t2, x0, finish      # jump to finish, if  t2 != 0
          add t0, t0, t1          # t0 = t0 + t1
          addi t1, t1, 2          # t1 = t1 + 2
          j loop                  # jump to loop
finish:   add a0, t0, x0          # a0 = t0
```

计算所有不大于 n 的奇数之和

HW2

Problem 2

Write a loop that reverses order of the bits of an 8-bit number in s0 and stores the result in s1. RISC-V registers have 4 bytes, but in this problem, the higher 24 bits of s0 and s1 are always 0. For example, if the lower 8 bits of s0 are 00001101, the lower 8 bites s1 should be 10110000.

```
pseudocode:
s1 = 0;                     //结果初始化为 0
for(t0=8; t0>0; t0--){
    t1 = s0 & 0x00000001    // t1 = s0 的最低位
    s0 = s0>>1              //s0 右移 1bit
    s1 = (s1<<1) + t1       //将 t1 添加到 s1 的最低位
}
```

```
begin:  addi t0, x0, 8          # t0 = 8
        addi s1, x0, 0          # s1 = 0
loop:   beq t0, x0, finish      # jump to finish, if t0 = 0
        andi t1, s0, 1          # t1 = s0 & 0x00000001
        srli s0, s0, 1          # s0 = s0 >> 1
        slli s1, s1, 1          # s1 = s1 << 1
        add s1, s1, t1          # s1 = s1 + t1
        addi t0, t0, -1         # t0 = t0 - 1
        jal x0, loop            # jump to loop
finish:
```


4

HW2

Problem 3

Assume we have an array in memory that contains int* arr = {1,2,3,4,5,6,0}. Let the values of arr be a multiple of 4 and stored in register s0. What do the snippets of RISC-V code do? Add comments to the following code.
Assume that all the instructions are run one after the other in the same context.

```
a)  lw      t0, 8(s0)          //Sets t0 equal to arr[2],  t0=3
b)  slli    t1, t0, 2          // t1=t0<<2=t0*4=12
    add  t2, s0, t1            // t2 = s0 + t1(首地址加偏移量)
    lw      t3, 0(t2)          // t3=arr[3]=4
    addi    t3, t3, 1          // t3=t3+1=5
    sw      t3, 0(t2)          // arr[3]=t3=5
Increments arr[t0 + 1] by 1
c)  lw      t0, 16(s0)         // t0=arr[4]=5
    xori    t0, t0, 0xFFF      // 按位异或,每一位都取反
    addi    t0, t0, 1          // t0=t0+1=-5
```

HW2

Problem 4

Write a function sumSquare in RISC-V that, when given an integer n, returns the summation below. If n is not positive, then the function returns 0.

$$n^2 + (n-1)^2 + (n-2)^2 + \cdots + 1^2$$

For this problem, you are given a RISC-V function called **square** that takes in an integer and returns its square. Implement **sumSquare** using square as a subroutine.

Solution

```
sumSquare:  addi sp, sp, -12
            sw ra, 0(sp) // 压栈 返回地址寄存器
            sw s0, 4(sp) // s0,s1 保存寄存器
            sw s1, 8(sp)
            add s0, a0, x0
            add s1, x0, x0
loop:       bge x0, s0, end // 0>=s0 go to end
            add a0, s0, x0 // a0=s0
            jal ra, square // 调用 square
            add s1, s1, a0 // 累加
            addi s0, s0, -1 // s0 中的值递减
            jal x0, loop
end:        add a0, s1, x0 // 结果放在 a0
            lw ra, 0(sp) // 出栈
            lw s0, 4(sp)
            lw s1, 8(sp)
            addi sp, sp, 12
            jr ra // 跳转到返回地址
```


浙江大学 信息与电子工程学院
Information Science & Electronic Engineering, Zhejiang University

HW3

Problem 1

There are 3 different processors P1, P2 and P3 with the same ISA, its clock frequencies and CPIs are as follow:

Processor	Clock Frequency	CPI
P1	2 GHz	1.5
P2	1.5 GHz	1.2
P3	3 GHz	2.0

1. Calculate the MIPS for each processor and find out which one has the best performance?
2. If we want to increase the MIPS by 30%, which will cause an increase of CPI by 15%, what's the clock frequency should be for each processor?

1. $$MIPS = \frac{\text{Clock Frequency}}{CPI} \times 10^{-6}$$

P1: 1333 MIPS
P2: 1250 MIPS
P3: 1500 MIPS
So, P3 has the best performance.

2. Above formula can be transformed to $\text{Clock Frequency} = MIPS \times CPI \times 10^{6}$

P1: $\text{Clock Frequency} = 1333 \times 1.3 \times 1.5 \times 1.15 \times 10^{6} \approx 2.99GHz$

P2: $\text{Clock Frequency} = 1250 \times 1.3 \times 1.2 \times 1.15 \times 10^{6} \approx 2.24GHz$

P3: $\text{Clock Frequency} = 1500 \times 1.3 \times 2 \times 1.15 \times 10^{6} \approx 4.49GHz$

HW3

Problem 2

The table below shows the instruction type breakdown of a given application executed on 1, 2, 4, or 8 processors. Using this data, you will be exploring the speedup of applications on parallel processors.

Processors	No. Instructions per processor			CPI		
	Arithmetic	Load/Store	Branch	Arithmetic	Load/Store	Branch
1	2560	1280	256	1	4	2
2	1280	640	128	1	5	2
4	640	320	64	1	7	2
8	320	160	32	1	12	2

1. The table above shows the number of instructions required per processor to complete a program on a multiprocessor with 1, 2, 4, or 8 processors. What is the total number of instructions executed per processor? What is the aggregate number of instructions executed across all processors? 合计

1.1 . a)

Processors	Instructions per processor	Total instructions
1	2560+1280+256 = 4096	4096
2	1280+640+128 = 2048	2048*2=4096
4	1024	4096
8	512	4096

HW3

Problem 2

The table below shows the instruction type breakdown of a given application executed on 1, 2, 4, or 8 processors. Using this data, you will be exploring the speedup of applications on parallel processors.

Processors	No. Instructions per processor			CPI		
	Arithmetic	Load/Store	Branch	Arithmetic	Load/Store	Branch
1	2560	1280	256	1	4	2
2	1280	640	128	1	5	2
4	640	320	64	1	7	2
8	320	160	32	1	12	2

1.2 Given the CPI values on the right of the table above, find the total execution time for this program on 1, 2, 4, and 8 processors. Assume that each processor has a 2 GHz clock frequency.

1.2 .a)

Processors	Execution Time
1	(2560*1+1280*4+256*2)/2G = 4.096*10^-6 s
2	2.368*10^-6 s
4	1.504*10^-6 s
8	1.152*10^-6 s

CPU时间 = 指令数 × CPI × 时钟周期时间

HW3

Problem 2

The table below shows the instruction type breakdown of a given application executed on 1, 2, 4, or 8 processors. Using this data, you will be exploring the speedup of applications on parallel processors.

Processors	No. Instructions per processor			CPI		
	Arithmetic	Load/Store	Branch	Arithmetic	Load/Store	Branch
1	2560	1280	256	1	4	2
2	1280	640	128	1	5	2
4	640	320	64	1	7	2
8	320	160	32	1	12	2

1.3 If the CPI of the arithmetic instructions was doubled, what would the impact be on the execution time of the program on 1, 2, 4, or 8 processors? Assume that each processor has a 2 GHz clock frequency.

1.3 . a)

Processors	Execution Time
1	(2560*2+1280*4+256*2)/2G = 5.376*10^-6 s
2	(1280*2+640*5+128*2)/2G = 3.008*10^-6 s
4	1.824*10^-6 s
8	1.312*10^-6 s

HW3

Problem 2

The table below shows the number of instructions per processor core on a multicore processor as well as the average CPI for executing the program on 1, 2, 4, or 8 cores. Using this data, you will be exploring the speedup of applications on multicore processors

Cores per Processor	Instructions per Core	Average CPI
1	1.00E+**10**	1.2
2	5.00E+**9**	1.4
4	2.50E+**9**	1.8
8	1.25E+**9**	2.6

1.4 Assuming a 3 GHz clock frequency, what is the execution time of the program using 1, 2, 4, or 8 cores?

1.4 .a)

Processors	Execution Time
1	(1.00*10^10*1.2)/3G = 4 s
2	(5.00*10^9*1.4)/3G = 2.33 s
4	1.5 s
8	1.08 s

运行时间=每个核的指令数*核数*CPI/核数*时钟频率

HW3

Problem 2

1.6 If using a single core, find the required CPI for this core to get an execution time equal to the time obtained by using the number of cores in the table above (execution times in problem 1.4). Note that the number of instructions should be the aggregate number of instructions executed across all the cores.

1.5 .a)

Processors	Required CPI
1	1.2
2	1.4/2= 0.7
4	1.8/4=0.45
8	2.6/8=0.325

$$\text{运行时间} = \frac{\text{总指令数} * \text{CPI}}{\text{时钟频率} * \text{核数}}$$

HW4

Problem 1

3.9 [10] <§3.2> Assume 151 and 214 are signed 8-bit decimal integers stored in two's complement format. Calculate 151 + 214 using saturating arithmetic. The result should be written in decimal. Show your work.

Answer：-128

HW4

Problem 2

3.27 [20] <§3.5> IEEE 754-2008 contains a half precision that is only 16 bits wide. The leftmost bit is still the sign bit, the exponent is 5 bits wide and has a bias of 15, and the mantissa is 10 bits long. A hidden 1 is assumed. Write down the bit pattern to represent -1.5625×10^{-1} assuming a version of this format, which uses an excess-16 format to store the exponent. Comment on how the range and accuracy of this 16-bit floating point format compares to the single precision IEEE 754 standard.

3.27 $-1.5625 \times 10^{-1} = -0.15625 \times 10^{0}$

$= -0.00101 \times 2^{0}$

move the binary point three to the right, $= -1.01 \times 2^{-3}$

exponent $= -3 = -3 + 15 = 12_{01100}$, fraction $= -0.0100000000$

answer: 1011000100000000

HW4

Problem 3

3.29 [20] <§3.5> Calculate the sum of 2.6125×10^{1} and $4.150390625 \times 10^{-1}$ by hand, assuming A and B are stored in the 16-bit half precision described in Exercise 3.27. Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps.

3.29 $2.6125 \times 10^{1} + 4.150390625 \times 10^{-1}$

$2.6125 \times 10^{1} = 26.125 = 11010.001 = 1.1010001000 \times 2^{4}$

$4.150390625 \times 10^{-1} = 0.4150390625 = 0.011010100111 = 1.1010100111 \times 2^{-2}$

Shift binary point six to the left to align exponents,

```
                GR
1.1010001000 00
1.0000011010 10 0111 (Guard 5 1, Round 5 0,
Sticky 5 1)
--------------------
1.1010100010 10
```

In this case the extra bit (G,R,S) is more than half of the least significant bit (0).

Thus, the value is rounded up.

$1.1010100011 \times 2^{4} = 11010.100011 \times 2^{0} = 26.546875 = 2.6546875 \times 10^{1}$

ISEE
浙大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

HW5

Problem 1

In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
250ps	400ps	150ps	350ps	200ps

Also, assume that instructions executed by the processor are broken down as follows:

alu	beq	lw	sw
40%	20%	25%	15%

a) What is the clock cycle time in a pipelined and non-pipelined processor?
b) What is the total latency of an LW instruction in a pipelined and non-pipelined processor?
c) If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

- a) pipelined: 400ps single-cycle : 250ps + 400ps + 150ps + 350ps + 200ps = 1350ps
- b) pipelined: 400ps*5=2000ps single-cycle : 1350ps
- c) ID 350ps

HW5

Problem 2

Look for data hazards in the code below, and figure out how forwarding could be used to solve them. them.

Instruction	C1	C2	C3	C4	C5	C6	C7
1. add t0, a0, -1	IF	ID	EX	MEM	WB		
2. and s2, t0, a0		IF	ID	EX	MEM	WB	
3. sltiu a0, t0, 5			IF	ID	EX	MEM	WB

Answer:

There are two data hazards, between instructions 1 and 2, and between instructions 1 and 3.

The first could be resolved by forwarding the result of the EX stage in C3 to the beginning of the EX stage in C4, and the second could be resolved by forwarding the result of the EX stage in C3 to the beginning of the EX stage in C5.

HW5

Problem 3

This exercise is intended to help you understand the relationship between forwarding, hazard detection, and ISA design. Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a 5-stage pipelined datapath:

```
add x5, x2, x1
lw x3, 4(x5)
or x3, x5, x3
lw x2, 0(x2)
sw x3, 0(x5)
```

1) if there is no forwarding or hazard detection, insert nop to ensure correct execution.
2) Repeat 1) but now use nop only when a hazard cannot be avoided by changing or rearranging these instructions. You can assume register x7 can be used to hold temporary values in your modified cade.
3) If the processor has forwarding, but we forget to implement the hazard detection unit, what happens when this code executes?

HW5

Problem 3

1) if there is no forwarding or hazard detection, insert nop to ensure correct execution.

1）

```
add x5, x2, x1
lw x3, 4(x5)
or x3, x5, x3
lw x2, 0(x2)
sw x3, 0(x5)
```

Instruction	C1	C2	C3	C4	C5	C6
1. add x5, x2, x1	IF	ID	EX	MEM	WB	
2. lw x3, 4(x5)		IF	ID	EX	MEM	WB

```
ADD x5, x2, x1
NOP
NOP
LW x3, 4(x5)
NOP
NOP
OR x3, x5, x3
LW x2, 0(x2)
NOP
SW X3, 0(x5)
```

Instruction	C1	C2	C3	C4	C5	C6
1. add x5, x2, x1	IF	ID	EX	MEM	WB	
2. NOP						
3. NOP						
4. lw x3, 4(x5)				IF	ID	MEM

HW5

2) Repeat 1) but now use nop only when a hazard cannot be avoided by changing or rearranging these instructions. You can assume register x7 can be used to hold temporary values in your modified cade.

2）

```
add x5, x2, x1
lw x3, 4(x5)
or x3, x5, x3
lw x2, 0(x2)
sw x3, 0(x5)
```

```
ADD x5, x2, x1
LW x2, 0(x2)
NOP
LW x3, 4(x5)
NOP
NOP
OR x3, x5, x3
NOP
NOP
SW X3, 0(x5)
```

HW5

```
add x5, x2, x1
lw x3, 4(x5)
or x3, x5, x3
lw x2, 0(x2)
sw x3, 0(x5)
```

3) If the processor has forwarding, but we forget to implement the hazard detection unit, what happens when this code executes?

3）With forwarding, the hazard detection unit is still needed because it must insert a one-cycle stall whenever the load supplies a value to the instruction that immediately follows that load. Without the hazard detection unit, the instruction that depends on the immediately preceding load gets the stale value the register had before the load instruction.

Instruction	C1	C2	C3	C4	C5	C6
lw x3, 4(x5)	IF	ID	EX	MEM	WB	
or x3, x5,x3		IF	ID	EX	MEM	WB

lw x3, 4(x5)
or x3, x5, x3

Instruction	C1	C2	C3	C4	C5	C6	C7
lw x3, 4(x5)	IF	ID	EX	MEM	WB		
NOP		IF	ID	EX	MEM	WB	
or x3, x5, x3			IF	ID	EX	MEM	WB

The register x3 in instruction “or x3, x5, x3” has the stale value instead of the value the load instruction (LW x3, 4(x5)) supplies.

取数-使用型数据冒险

HW5

Problem 4

In the conventional fully bypassed 5-stage pipeline discussed in class, the main remaining data hazard is the load-use hazard. For example, in the following instruction sequence:

```
lw x1, 16(x2)
xor x3, x1, x5
```

The xor instruction will experience a one-cycle stall in the decode stage, while the lw propagates to the memory stage. For the remainder of this question, we will only consider 32-bit loads (lw). ***You may ignore branch and jump instructions in this problem.***

```
F   D   X   M   W                       lw
    F   D   D   X   M   W               xor
```

One approach to removing the hazard is to speculate on the load value returned (*load-value speculation*). In lecture, we briefly described one scheme, a *load-zero predictor* where the value was predicted to be zero. An instruction that would otherwise stall in the decode stage waiting for a memory value was provided the value 0 instead. When the load instruction reaches the memory stage, the value is checked and if not zero, the pipeline is flushed and the dependent instruction is replayed as shown.

```
F   D   X   M   W                       lw
    F   D   X   -   -                   xor flushed
        F   D   -   -   -               flushed
            F   -   -   -   -           flushed
                F   D   X   M   W       xor correct value
```

HW5

Problem 4

Q3.A [3 points] Assuming the pipeline is flushed as shown on a mispredict, calculate the minimum accuracy required for the load-zero predictor to improve performance. Note the predictor is only used when an instruction would otherwise stall in decode waiting for a load value.

Without load-zero predictor
Needs one cycle stall on the decode stage.
Totally 7 cycles.

F	D	X	M	W				lw
	F	D	D	X	M	W		xor

设α为最小预测准确率

$$6\alpha + 9(1-\alpha) = 7$$

$$\alpha = \frac{2}{3} \approx 66.7\ \%$$

With load-zero predictor
(predict correctly) 6 cycles.
(predict incorrectly) 9 cycles.

F	D	X	M	W					lw
	F	D	X	-	-				xor flushed
		F	D	-	-	-			flushed
			F	-	-	-	-		flushed
				F	D	X	M	W	xor correct value

Mispredict penalty of a predicted load-zero is 3 cycles. The load-use delay is 1 cycle. The predictor should be at least 66.7% accurate to improve performance.

信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

HW6

Problem1

In a **direct-mapped cache**, each memory address is associated with one possible block within the cache. The memory address is divided into three fields, as shown below.

Tag	Index	(Byte) Offset

Suppose we have a 128Bytes of data in a direct-mapped cache with 16-byte blocks. Determine the size of the Tag, Index and (Byte) Offset fields if the memory address is 32-bit.

Answer:

The cache has 16-byte block , so the Offset is 4 bits (2^4 = 16).

The cache contains 128Bytes data (2^7 Bytes), and block contains 16 Bytes, so the direct-mapped cache has 8 (128/16) blocks. The Index needs 3 (2^3 = 8) bits.

Tag length = 32 – Index bits – Offset bits = 32 – 3 -4 = 25 bits

index: cache大小为2^n个块，因此n位被用来索引

offset: 块大小为2^m个字（$2^{(m+2)}$个字节），因此m位用来查找块中的字，两位是字节偏移消息

tag: 标记域的大小为 $32-(n+m+2)$

HW6

Problem2

For a direct-mapped cache design with a 32bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31-13	12-6	5-0

a)What is the cache line size(in words)?

b)How many entries does the cache have?

c)What is the ratio between total bits required for such a cache implementation over the data storage bits? (Note the structure of the direct-mapped cache!)

Answer:

a) RISC-V 32位来说， 1 word = 32 bits = 4 bytes
cache line(block) size: 2^(Offset bits) = 2^6 = 64 bytes = 16 words

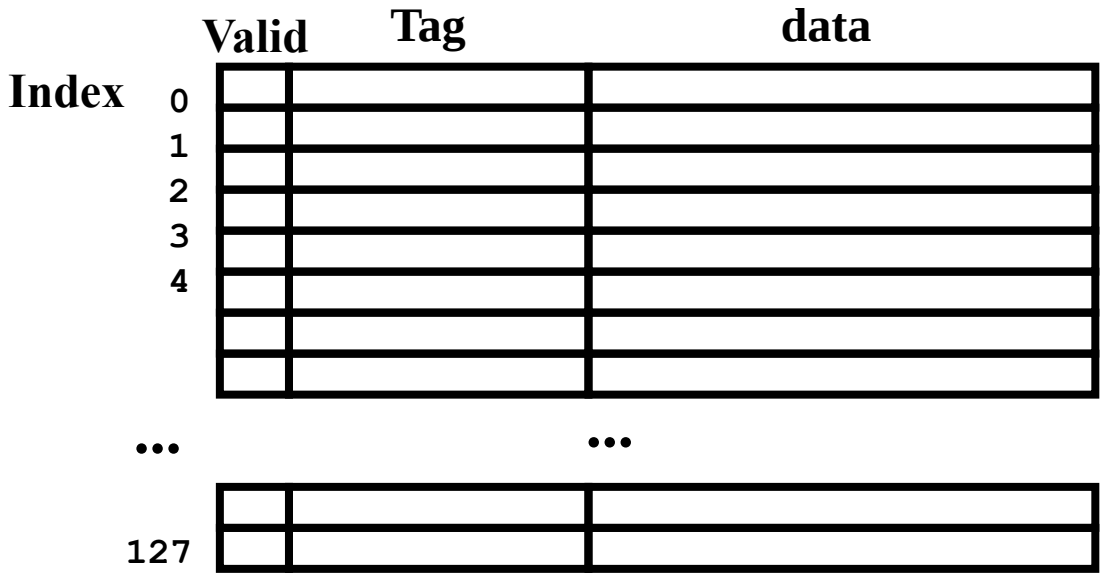
b) The number of entries: 2^(Index bits) = 2^7 = 128

HW6

Problem2

c)What is the ratio between total bits required for such a cache implementation over the data storage bits? (Note the structure of the direct-mapped cache!)

The structure of cache is shown as follows:



c) (Valid bits + Tag bits + data bits) / (data bits)

= (1 + 19 + 16*32) / 16*32 = 1.039

HW6

Problem3

Cache is important to providing a high-performance memory hierarchy to processors. Below is a list of 32-bit memory address references, given as **word address.**

3, 180, 43, 2, 181, 88, 164, 180, 89.

a) For each of these references, identify the binary address, the tag, and the index given a direct-mapped cache with **8 two-word blocks**. Also list if each reference is a hit or a miss, assuming the cache is initially empty.

1 word = 4 bytes-------block大小=8bytes-------offset=3bits

cache有8个------index=3

注意：给的是word address，因此需要转换为byte，*4即补00

HW6

Problem3

a) For each of these references, identify the binary address, the tag, and the index given a direct-mapped cache with **8 two-word blocks**. Also list if each reference is a hit or a miss, assuming the cache is initially empty.

Word address	Binary address	Tag	Index	Hit/Miss
3	0000 0011	0000	001	M
180	1011 0100	1011	010	M
43	0010 1011	0010	101	M
2	0000 0010	0000	001	H
181	1011 0101	1011	010	H
88	0101 1000	0101	100	M
164	1010 0100	1010	010	M
180	1011 0100	1011	010	M
89	0101 1001	0101	100	H

index has 3 bits. offset has 3 bits.

Word address: 3（0000 0011）

	Tag	index	offset
byte address:	0000	001	100

HW6

Problem3

b) For each of these references, identify the binary address, the tag, and the index given a direct-mapped cache with four-word blocks and a total size if 4 blocks. Also list if each reference is a hit or a miss, assuming the cache is initially empty.

Word address	Binary address	Tag	Index	Hit/Miss
3	0000 0011	0000	00	M
180	1011 0100	1011	01	M
43	0010 1011	0010	10	M
2	0000 0010	0000	00	H
181	1011 0101	1011	01	H
88	0101 1000	0101	10	M
164	1010 0100	1010	01	M
180	1011 0100	1011	01	M
89	0101 1001	0101	10	H

1 word = 4 bytes-------block大小=16bytes-------offset=4bits

cache有4个blocks------index=2

Word address: 3 binary address: 0000 0011

	Tag	index	offset
byte address:	0000	00	1100

HW6

Problem4

This exercise examines the impact of cache designs. For this exercise, the table of word address streams is shown in below.

1, 134, 212, 1, 135, 213, 162, 161, 2, 44, 41, 221

What is the miss rate for a fully associative cache with two-word blocks and a total size of 8 words, using LRU replacement? (LRU, Least recently use)

1 word = 4 bytes-------block大小=8bytes-------offset=3bits

cache有4个------index=2

注意：给的是word address，因此需要转换为byte，*4即补00

全相联（fully associative）：一个块可以放置在cache中的任何位置。需要检索所有的项，所以只适用于块数较少的cache

LRU：替换最久没有使用的块

HW6

Problem4

This exercise examines the impact of cache designs. For this exercise, the table of word address streams is shown in below.

1, 134, 212, 1, 135, 213, 162, 161, 2, 44, 41, 221

What is the miss rate for a fully associative cache with two-word blocks and a total size of 8 words, using LRU replacement? (LRU, Least recently use)(75%)

word address	binary address	block address	block address	hit/miss	block0	block1	block2	block3
1	00000001	0000000	0	miss	**Mem[1]**			
134	10000110	1000011	67	miss	Mem[1]	**Mem[134]**		
212	11010100	1101010	106	miss	Mem[1]	Mem[134]	**Mem[212]**	
1	00000001	0000000	0	hit	**Mem[1]**	Mem[134]	Mem[212]	
135	10000111	1000011	67	hit	Mem[1]	**Mem[134]**	Mem[212]	
213	11010101	1101010	106	hit	Mem[1]	Mem[134]	**Mem[212]**	
162	10100010	1010001	81	miss	Mem[1]	Mem[134]	Mem[212]	**Mem[162]**
161	10100001	1010000	80	miss	**Mem[161]**	Mem[134]	Mem[212]	Mem[162]
2	00000010	0000001	1	miss	Mem[161]	**Mem[2]**	Mem[212]	Mem[162]
44	00101100	0010110	22	miss	Mem[161]	Mem[2]	**Mem[44]**	Mem[162]
41	00101001	0010100	20	miss	Mem[161]	Mem[2]	Mem[44]	**Mem[41]**
221	11011101	1101110	110	miss	**Mem[221]**	Mem[2]	Mem[44]	Mem[41]

此时没有空闲且未命中，替换最少使用的block0

息与电子工程学院
ronic Engineering, Zhejiang University

HW6

Problem5

You are trying to reverse-engineer the characteristics of a cache in a system, so that you can design a more efficient, machine-specific implementation of an algorithm you are working on. To do so, you have come up with three patterns that access various bytes in the system in an attempt to determine the following four cache characteristics:

- Cache block size (8, 16, 32, 64, or 128 B)
- Cache associativity (1-, 2-, 4-, or 8-way)
- Cache size (4 or 8 KB)
- Cache replacement policy (LRU or FIFO)

However, the only statistic that you can collect on this system is cache hit rate after performing the
access pattern. Here is what you observe (the address is **in bytes**):

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

HW6

Problem5

Assume that the cache is initially **empty** **at the beginning of the first sequence**, but not at the beginning of the second and third sequences. The sequences are executed back-to-back, i.e., no other accesses take place between the three sequences. **Thus, at the beginning of the second (third) sequence, the contents are the same as at the end of the first (second) sequence.**

Based on what you observe, what are the following characteristics of the cache? **Explain the reason**.

(a) Cache block size (8, 16, 32, 64, or 128 B)?
(b) Cache associativity (1-, 2-, 4-, or 8-way)?
(c) Cache size (4 or 8 KB)?
(d) Cache replacement policy (LRU or FIFO)?

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

(a) Cache block size (8, 16, 32, 64, or 128 B)?

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
1000000000000	8192	M
111111	63	H
10000000000000	16384	M
1000000000000	4096	M
1000000000000	8192	H
1000000	64	H
10000000000000	16384	H

注意：给出的是字节地址，不需要补00

序列1：
命中率为0.5，即命中3次
hit一定是4.8.16
Offset=5或6bit

序列2：
命中5次，miss3次
Miss一定是8912.16384.4096

等效要求64第一次miss，63第一次hit
Offset=6bit　Block大小为64B

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

(b) Cache associativity (1-, 2-, 4-, or 8-way)

	0	0	M
	100	4	H
	1000	8	H
	10000	16	H
	1000000	64	M
	10000000	128	M
	11111	31	H
1000000000000		8192	M
	111111	63	H
1000000000000		16384	M
1000000000000		4096	M
1000000000000		8192	H
	1000000	64	H
1000000000000		16384	H

分别讨论4种情况，发现只有4路满足要求

Cache size最大为8KB

1 way: index最多为7位
每个set有1个block
在0miss之后，会被放入set0，此时tag为0
在8192miss之后，会被放入set0，替换0，此时tag为1
那么63一定会miss
不满足要求

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

(b) Cache associativity (1-, 2-, 4-, or 8-way)

	0	0 M
	100	4 H
	1000	8 H
	10000	16 H
	1000000	64 M
	10000000	128 M
	11111	31 H
100000000000000		8192 M
	111111	63 H
100000000000000		16384 M
1000000000000		4096 M
100000000000000		8192 H
	1000000	64 H
100000000000000		16384 H

Cache size最大为8KB

2 way: index最多为6位
每个set有2个block
在0miss之后，会被放入set0，此时tag为0
在8192miss之后，会被放入set0，此时tag为10
在16384miss之后，会被放入set0，因为block只有两个，因此会替换0或8192，并改变tag
在16384miss之后，会被放入set0，因为block只有两个，因此会替换0或8192或16384，并改变tag
那么后续的8192或16384一定会miss
不满足要求

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

(b) Cache associativity (1-, 2-, 4-, or 8-way)

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
10000000000000	8192	M
111111	63	H
100000000000000	16384	M
10000000000000	4096	M
10000000000000	8192	H
1000000	64	H
100000000000000	16384	H
100000000000000	32768	M
0	0	
10000001	129	H
10000000000	1024	M
11000000000	3072	M
10000000000000	8192	

Cache size最大为8KB

8 way: index最多为4位

每个set有8个block

在0miss之后，会被放入set0，此时tag为0

在8192miss之后，会被放入set0，此时tag为1000

在16384miss之后，会被放入set0，此时tag为10000

在4096miss之后，会被放入set0，此时tag为100

在32768miss之后，会被放入set0，此时tag为100000

序列3的0：hit

序列3的129：hit（前面已经有128）

1024miss，会被放入set0，此时tag为1

3072miss，会被放入set0，此时tag为11

序列3的8192：hit（前面已经有8192且未被替换）

此时序列3的命中率为0.5

不满足要求

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

(b) Cache associativity (1-, 2-, 4-, or 8-way)

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
10000000000000	8192	M
111111	63	H
100000000000000	16384	M
10000000000000	4096	M
10000000000000	8192	H
1000000	64	H
100000000000000	16384	H
100000000000000	32768	M
0	0	
10000001	129	H
10000000000	1024	M
110000000000	3072	M
10000000000000	8192	

Cache size为8KB

4 way: index为5位

每个set有4个block

在0miss之后，会被放入set0，此时tag为0

在8192miss之后，会被放入set0，此时tag为100

在16384miss之后，会被放入set0，此时tag为1000

在4096miss之后，会被放入set0，此时tag为10

在32768miss之后，会被放入set0，此时block已满，则替换0/8192/16384/4096，此时tag为10000

注意：无论是FIFO还是LRU，都会替换0

序列3的0：miss，放入set0，替换8192/16384/4096/32768

序列3的129：一定hit（前面已经有128且未被替换）序列3的8192：hit，已保证命中率为0.33

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
100000000000	8192	M
111111	63	H
1000000000000	16384	M
100000000000	4096	M
100000000000	8192	H
1000000	64	H
1000000000000	16384	H
1000000000000	32768	M
0	0	
10000001	129	H
10000000000	1024	M
110000000000	3072	M
100000000000	8192	

(c) Cache size (4 or 8 KB)? answer:8KB

Cache size为4KB

4 way: index最多为4位

每个set有8个block

在0miss之后，会被放入set0，此时tag为0

在8192miss之后，会被放入set0，此时tag为1000

在16384miss之后，会被放入set0，此时tag为10000

在4096miss之后，会被放入set0，此时tag为100

在32768miss之后，会被放入set0，此时block已满，则替换0/8192/16384/4096，此时tag为10000

注意：无论是FIFO还是LRU，都会替换0

序列3的0：miss，替换8192/16384/4096/32768

序列3的129：一定hit（前面已经有128且未被替换）

序列3的1024和3072，miss之后也会被放入set0，因此替换两个

注意：无论是FIFO还是LRU，都会替换8192

序列3的8192：miss，不满足要求

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
10000000000000	8192	M
111111	63	H
100000000000000	16384	M
1000000000000	4096	M
10000000000000	8192	H
1000000	64	H
100000000000000	16384	H
1000000000000000	32768	M
0	0	
10000001	129	H
10000000000	1024	M
110000000000	3072	M
1000000000000	8192	

(d) Cache replacement policy (LRU or FIFO)?LRU

LRU：满足条件

0			
0	8192		
0(hit)	8192		
0	8192	16384	
0	8192	16384	4096
0	8192(hit)	16384	4096
0	8192	16384	4096
0	8192	16384(hit)	4096
32768	8192	16384	4096
32768	8192	16384	0
32768	8192	16384	0
32768	8192	16384	0
32768	8192	16384	0
32768	8192(hit)	16384	0

工程学院 Zhejiang University

HW6

Sequence	Addresses Accessed (Oldest → Youngest)								Hit Rate
1.	0	4	8	16	64	128			1/2
2.	31	8192	63	16384	4096	8192	64	16384	5/8
3.	32768	0	129	1024	3072	8192			1/3

0	0	M
100	4	H
1000	8	H
10000	16	H
1000000	64	M
10000000	128	M
11111	31	H
10000000000000	8192	M
111111	63	H
100000000000000	16384	M
1000000000000	4096	M
10000000000000	8192	H
1000000	64	H
100000000000000	16384	H
100000000000000	32768	M
0	0	
10000001	129	H
10000000000	1024	M
110000000000	3072	M
10000000000000	8192	

(d) Cache replacement policy (LRU or FIFO)?LRU

FIFO：不满足条件

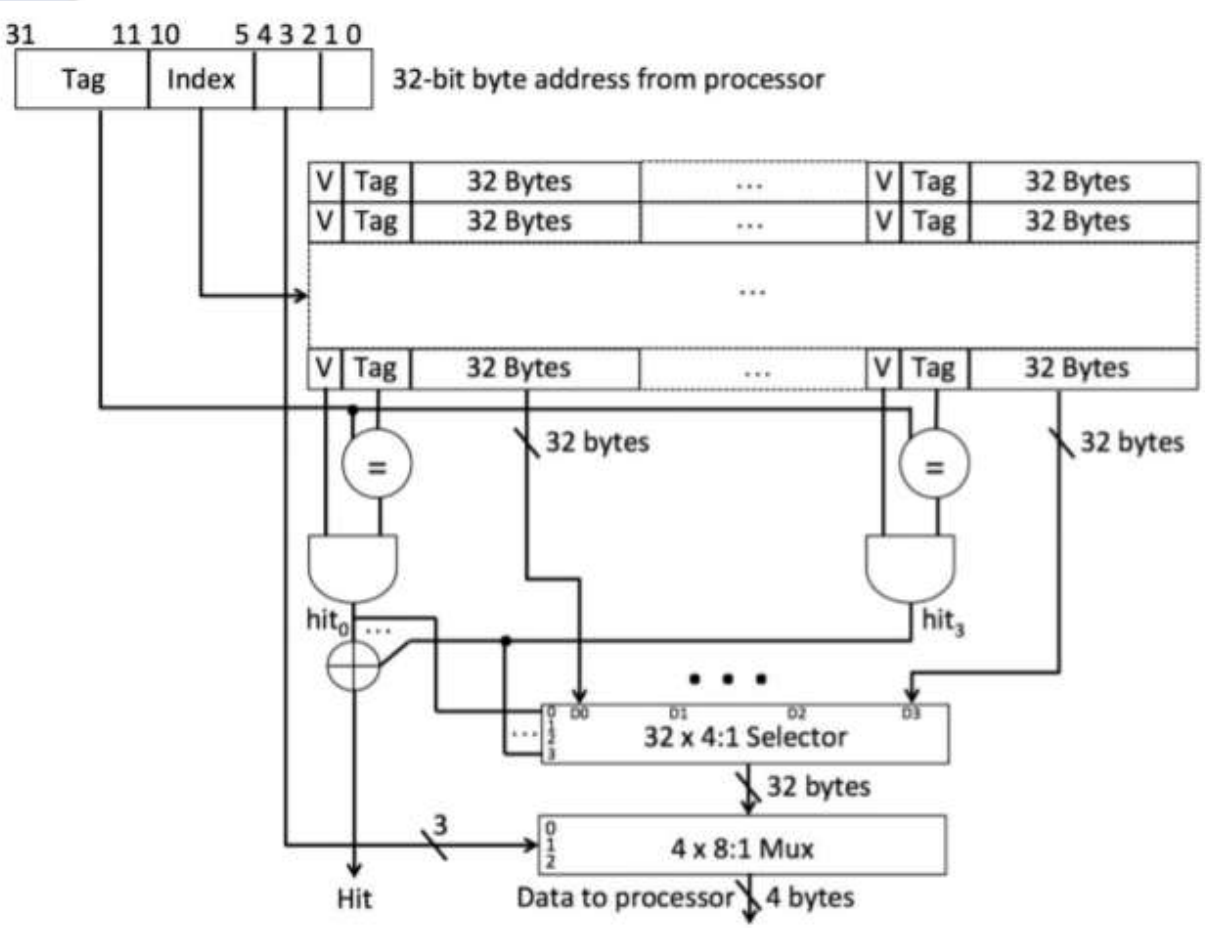
因为序列3中0和8192都会miss

0			
0	8192		
0(hit)	8192		
0	8192	16384	
0	8192	16384	4096
0	8192(hit)	16384	4096
0	8192	16384	4096
0	8192	16384(hit)	4096
32768	8192	16384	4096
32768	0	16384	4096
32768	0	16384	4096
32768	0	16384	4096
32768	0	16384	4096
32768	0	16384	4096

HW6

Problem6

You are given the sketch of a cache design below:

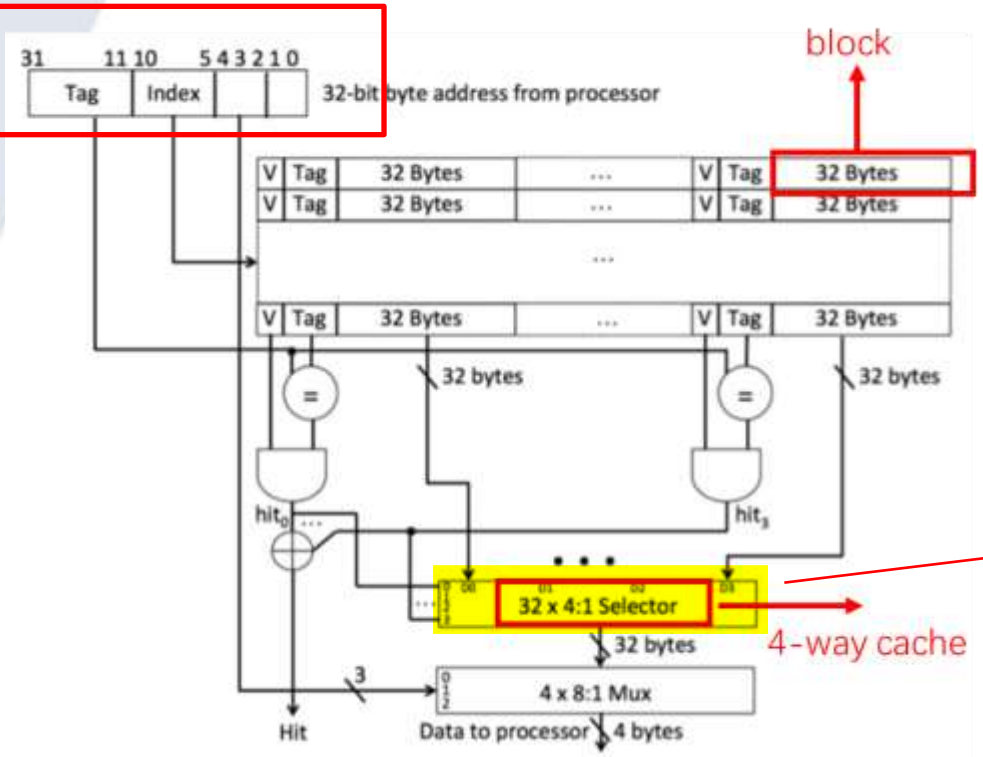


Note: a 4:1 Selector has four control inputs labeled 0, 1, 2, 3 and four data inputs D0, D1, D2, D3. It connects Di to the output when the hit control signal is true. Assume that at most one control input is true at any time. The selector function is undefined when none of the control inputs are true.

Answer the following questions about the cache above.

HW6

cache



a) What is the block size of the cache in bytes?

32 Bytes

b) What is the number of blocks in this cache?

blocks 32 bytes, offset 5bits (4~0bit)

index 10~5 bit, index has 6bits.

the cache has 2^6 = 64 entries , each entry has 4 blocks (4-way cache)

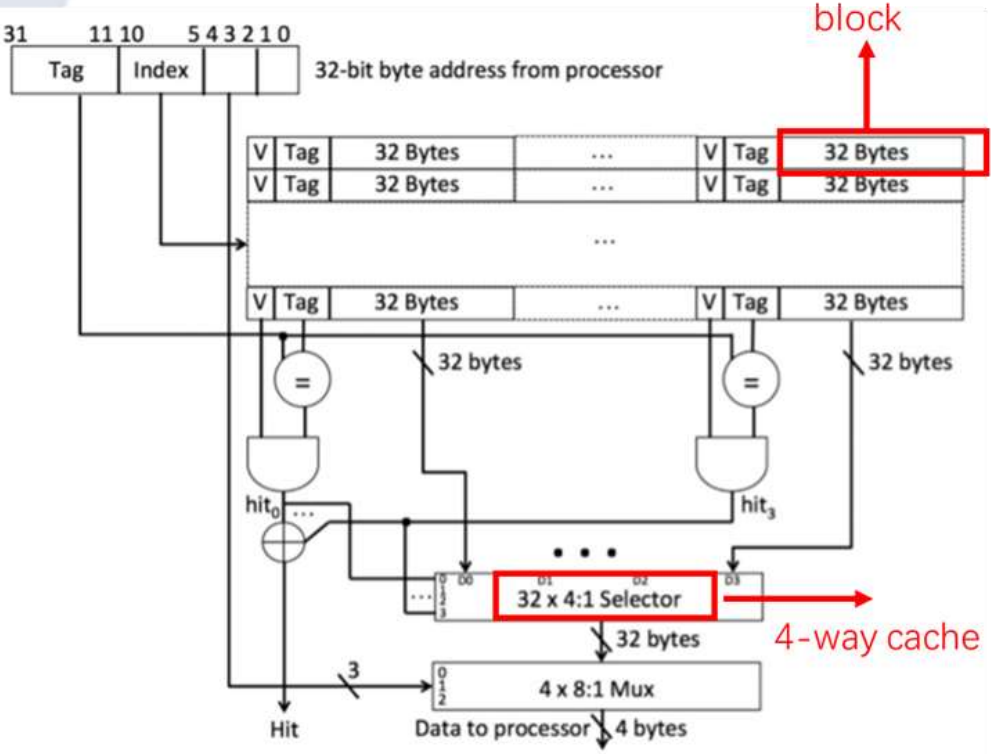
The number of blocks is 64 * 4 = 256

c) What is the total data capacity of the cache in bytes?

data capacity = blocks number * blocks size

256 * 32 = 8192 bytes.

HW6



d) What is the associativity of the cache?

4:1 selector 4-way

e) Is this a write-through or write-back cache?

write-through

There is no dirty bit, so it is a write-through cache.

f) What is the total number of valid bits in the cache?

256

There are 256 blocks, and each block needs 1 valid bit. Thus there are 256 valid bits.

g) What is the total number of tag bits in the cache?

5376

Each tag needs 21 bits, and each block needs one tag. Thus the tag bits in the cache is

HW6

Problem7

Caches are important to providing a high performance memory hierarchy to process. Below is a list of 32bit memory address references, **given as word address.**

7, 215, 175, 215, 7, 84, 65, 174, 64, 105, 85, 215

a) You're asked to optimize a cache design for the given references. There are three direct-mapped cache designs possible, all with total of 8 words of data:

C1 has 1-word blocks, C2 has 2-word blocks, and C3 has 4-word blocks.

In term of miss rate, which cache design is the best?

If the miss stall time is 25 cycles, and C1 has an access time of 2 cycles, C2 takes 3 cycles, and C3 takes 5 cycles, which is the best cache design?

HW6

address	Binary address	C2	C3	H/M C1
7	0000 0111	M	M	M
215	1101 0111	M	M	M
175	1010 1111	M	M	M
215	1101 0111	M	M	M
7	0000 0111	M	M	M
85	0101 0101	M	M	M
65	0100 0001	M	M	M
174	1010 1110	M	M	M
64	0100 0000	H	H	M
105	0110 1001	M	M	M
84	0101 0100	H	M	M
215	1101 0111	M	M	M

解：因为总共有 8words数据，那么：

C1：8words/1word=8 blocks　C2：8words/2word=4 blocks　C3：8words/4word=2 blocks

根据直接映射方式，可以得到三个cache的命中次数，当Hit的时候时间是access时间，当miss的时候，会引起阻塞时间25 cycles，所以：

总时间 = Hit rate* Hit time + Miss rate*(Hit time + Stall time)

= (Hit rate + Miss rate)*Hit time + Miss rate * Stall time = Hit time + Miss rate*Stall time

C1：0hit，stall time=25*12+2*12=324 cycles

C2：2hit，stall time=25*10+3*12=286 cycles

C3：1hit，stall time=25*11+5*12=335 cycles

C2 best